DEEP RESEARCH REPORT

Magnetar vs. Jolt-Akita: Closing the Proof-Size Gap

Proof-size reduction, recursion, folding, IVC, and PCD for hash-based zkVMs — a quantitative analysis of the 2024–2026 literature, with a structural cost model, a three-horizon roadmap, and a competitive verdict against the lattice frontier.

Z.ai Deep Research

Grounded in: Magnetar (2026-2423); Akita (LayerZero Labs & a16z crypto research); BinarySpartan 2026/1656; Ligerito 2025/1187; Twist & Shout 2025/105; and the folding literature from Nova 2021/370 to ProtogaLattice 2026/1317 and PikkuFold 2026/1809.

TECHNICAL SURVEY · OCTOBER 2026

VERSION 1.0

Table of Contents

1. Executive Summary 1

2. The Competitive Battleground: Magnetar vs. Jolt-Akita Today 2

2.1 Head-to-head baseline . . . 2

2.2 What Akita actually is, and where it is weak . . . 3

2.3 Akita's benchmark of the hash-based world . . . 4

3. Why Hash-Based Proofs Are Big: A Structural Cost Model 5

3.1 Anatomy of Magnetar's 598 KiB . . . 5

3.2 Why the floor is where it is . . . 6

4. Lever 1: Recursive Composition with Dual-Rate Proving 7

4.1 The strategy, made precise . . . 7

4.2 Quantifying the recursion overhead . . . 8

4.3 Aggregation before the wrap . . . 8

5. Lever 2: PCS-Layer Optimization (Codes, Rates, Fields) 9

5.1 What Ligerito's flexibility permits . . . 9

5.2 Code switching for commitment speed . . . 10

5.3 Sparsity-aware commitment of one-hot data . . . 10

5.4 Alternative backends and their caveats . . . 10

6. Lever 3: Protocol-Layer Size Reduction 11

6.1 Commit fewer bits . . . 11

6.2 Zero-knowledge as a cheap add-on . . . 12

7. Lever 4: Proving Throughput (Cycles per Second) 12

8. The Folding, IVC, and PCD Landscape for a Hash-Only zkVM 13

8.1 Why classical folding does not apply - and what does . . . 13

8.2 IVC architecture decision for Magnetar . . . 14

8.3 PCD and the distributed Fiat-Shamir problem . . . 15

9. The Hybrid Option: Adding a Lattice Layer 15

10. Quantified Roadmap and Competitive Outlook 16

10.1 Risk register . . . 17

10.2 The verdict . . . 18

11. Conclusions and Recommendations 18

Appendix: Sources and Attribution 19

1. Executive Summary

Magnetar and Jolt-Akita are the two fastest post-quantum zkVM provers measured to date, and they embody opposite bets on where post-quantum security should come from. Magnetar, described by Setty in October 2026, grounds every assumption in standard hash functions: its witness lives over binary fields, its single commitment is hash-based (Ligerito), and its 120-bit security bound is fully proven, relying only on Reed-Solomon proximity gaps within the unique-decoding radius. Jolt-Akita swaps the commitment layer for Akita, a Module-SIS lattice polynomial commitment scheme from LayerZero Labs and a16z crypto research, obtaining 61-70 KB openings and end-to-end Jolt proofs under 100 KB. This report examines, in depth, how Magnetar can close the proof-size gap to Akita while extending its existing throughput lead, drawing on the full 2024-2026 literature on folding, IVC, PCD, recursion, and binary-field proving.

5-18x
Magnetar proving throughput lead over Jolt-Akita at 2^22 cycles (geomean about 8.5x)

598 vs 91 KiB
proof-size gap today: the single axis on which Akita clearly wins

10-12 ms
Magnetar verification at 2^22 cycles, vs 15.6-39.8 ms for Jolt-Akita at 2^23-2^27

~100-150 KiB
projected Magnetar final proof after dual-rate recursion (Horizon 2)

The competitive picture is asymmetric. On speed, Magnetar already dominates: 6.5-7.7 million cycles per second across 15 workloads, versus Jolt-Akita at roughly 1.48 million padded rows per second at its best trace length of 2^27 - an application-level factor of 5-18x that persists even when Jolt-Akita runs on traces four times longer. On verification time the two systems are comparable, with Magnetar faster at its 2^22-2^24 operating point. On memory, Magnetar's planned chunked recursion bounds footprint while Akita's own paper concedes its proving time is "not yet on par" with its lattice predecessors. Neither system has shipped end-to-end recursive composition: Magnetar's recursion is a design proposal, and Jolt-Akita's is a merged-but-unfinished verifier-guest pull request. The one decisive Akita advantage is proof size, and it is structural: a lattice commitment proves evaluations with a logarithmic number of ring elements, while a hash-based commitment pays Merkle authentication paths for every query.

Our central finding is that this gap is largely, though not entirely, closable without abandoning Magnetar's hash-only purity. The proof-size cost concentrates in a single place - the one Ligerito evaluation argument that closes the protocol - so three independent levers attack it directly: (1) parameter and code tuning of the commitment layer (Horizon 1, roughly 240-350 KiB); (2) the dual-rate recursive composition Magnetar already sketches - prove application chunks at a high code rate for speed, then prove the small verifier circuit once at a low rate for size (Horizon 2, roughly 100-160 KiB); and (3) aggressive measures - 64-bit challenge fields, lower rates accepting prover slowdown, or a lattice final wrap - to push below 100 KiB (Horizon 3). In parallel, four throughput levers (small-domain sum-check arithmetic, Bolt-style sketched codes, streaming small-space proving, and GPU offload) project Magnetar from 6.9 to a plausible 10-20+ million cycles per second. The folding-scheme revolution surveyed in this report - Nova through ProtogaLattice and PikkuFold - turns out to be strategically irrelevant to a hash-only system, because folding requires linearly homomorphic commitments; the correct hash-only instruments are recursion, aggregation before the final wrap, and PCD

trees for parallelism.

6.6x

the proof-size factor Magnetar must close to reach parity with Jolt-Akita - and the factor Akita must close on proving speed to reach parity with Magnetar

The report proceeds as follows. Section 2 establishes the head-to-head competitive baseline from both systems' own measurements. Section 3 builds a structural cost model explaining why hash-based proofs are large and where exactly Magnetar's 598 KiB goes. Sections 4-7 analyze the four levers on proof size and proving speed in depth. Section 8 surveys folding, IVC, and PCD and derives the architecture decision for a hash-only zkVM. Section 9 honestly prices the hybrid option of adopting a lattice layer. Section 10 quantifies the three-horizon roadmap with a risk register, and Section 11 closes with prioritized recommendations. All projections are labeled as estimates with stated assumptions; all measured numbers are attributed to their source papers.

2. The Competitive Battleground: Magnetar vs. Jolt-Akita Today

2.1 Head-to-head baseline

Both teams benchmarked on the same Apple M4 Max hardware (Magnetar using 12 performance cores and 128 GiB, the Akita paper using 16 cores and 64 GiB), which makes the comparison unusually clean for preprint-era systems. Table 1 assembles the side-by-side picture from each paper's own reported numbers. Where the papers measure slightly different things (Magnetar's proofs at 2^22 application cycles; Jolt-Akita's at 2^17-2^27 padded trace rows), we note it in the table rather than silently harmonizing.

Metric	Magnetar (hash-only)	Jolt-Akita (lattice)
Proving throughput	6.5-7.7M cycles/s at 2^22; 5-18x Jolt-Akita app-level (geomean 8.5x)	about 1.48M padded rows/s at 2^27 (SHA-256 chains, 90.85 s)
Proof size	530-598 KiB at 2^20-2^22 cycles	85.9-98.1 KB over 2^17-2^27; 61-70 KB standalone openings
Verification time	8-12 ms at 2^16-2^22 cycles	9.96-39.82 ms over 2^17-2^27
Prover memory	0.2-6.4 GiB at 2^16-2^22; about 1.5 KiB/cycle at 2^24 (25 GiB)	0.33-35.5 GiB over 2^17-2^27; leaner per padded row at large T
Security level	120-bit proven; unique-decoding RS proximity gaps only	Module-SIS sized near 128 bits; sum-check over 128-bit field leaves overall level below 128
Underlying assumptions	SHA-256 or SHA-3 only; transparent; no trusted setup	Module-SIS (worst-case connected); ROM Fiat-Shamir, QROM analysis open
ZK support	None today (masking is standard future work)	None today (stated future work)
End-to-end recursion	Designed, not implemented (chunk + self-verification plan)	In progress: Akita field-inline verifier guest, PR #1911, under 2^26 rows

Table 1. Magnetar vs. Jolt-Akita, from each system's own evaluation

Three observations sharpen the framing. First, Magnetar's speed advantage is not a marginal engineering lead but a factor-of-several structural one, and it holds at every trace length measured: even at Jolt-Akita's best length (2^27), Magnetar at its best (2^24, the largest chunk it expects to use) remains 3.2-7.8x ahead on the three workloads Jolt itself reports. Second, the verification-time race is effectively tied at Magnetar's operating point - Magnetar verifies a 2^22-cycle proof in 10-12 ms while Jolt-Akita needs 15.6 ms at 2^23 and 39.8 ms at 2^27 - which matters because verifier cost is precisely the recursion workload. Third, both teams arrive with an unfinished recursion story, so the decisive battles (final proof size and unbounded-trace scaling) will be fought on territory neither side has fully occupied yet.

2.2 What Akita actually is, and where it is weak

Akita is a polynomial commitment scheme, not a zkVM: Jolt remains the machine, and Akita replaces the Dory elliptic-curve commitment beneath it. Its design history matters for predicting its trajectory. The lattice PCS family faced a trilemma - small proofs, fast verification, and standard Module-SIS security, of which prior systems achieved at most two. Greyhound and Hachi proved small (59-67 KB) but verified in time square-root in the polynomial size; RoKoKo and Grand Danois reached polylogarithmic verification only by leaning on stronger, less-studied assumptions (vanishing SIS), with Grand Danois landing at roughly 80 KB for 2^32-sized evaluations; LaBRADOR-derived toolkits verify in linear time. Akita breaks the trilemma with setup offloading: public setup matrices are committed in advance, the verifier's work on them is deferred and proven against those commitments, and iterating this fold yields a K-th-root verifier for any constant K - concretely under 20 ms single-threaded even at a 2^30-sized polynomial. Commitments compress to 128 bytes each, and the scheme offers pay-per-bit commitment cost: committing to a sparse vector costs only its support, which is decisive for Twist and Shout's one-hot address polynomials that are

almost entirely zero.

Akita's own conclusion section is a candid list of weaknesses an opponent should note. Proving time is "not yet on par with Greyhound and RoKoKo" - at 2^35 bits, direct Akita commits in 24.6 s and opens in 16.1 s, against hash-based systems like Flock-Ligerito that open in 5.2 s. The Fiat-Shamir transformation is analyzed only in the classical random oracle model, with the quantum ROM proof explicitly future work - a meaningful gap for a system whose raison d'etre is post-quantum security. Zero-knowledge is absent. And the response-model premise used for optimized later-fold schedules is heuristic rather than proven. None of these are fatal - the same could be said of most 2026-era systems, including Magnetar's unimplemented recursion - but they define the moving target: Akita will get faster to prove and smaller, so a static Magnetar advantage erodes.

2.3 Akita's benchmark of the hash-based world

The most useful single table in the Akita paper is its independent measurement of the hash-based PCS competition at 2^31 bits committed, reproduced in abbreviated form in Table 2. Two facts jump out. First, every hash-based proof at that scale is 7-14x larger than Akita's 63-68 KB - Magnetar's proof-size problem is the entire hash-based family's problem, not a Magnetar-specific defect. Second, the hash-based systems verify faster than Akita (0.8-3.3 ms versus 5.6-12.7 ms), because verifying a few dozen hashes beats lattice ring arithmetic - except SP1's BaseFold at 20.3 ms. The fastest verifiers in the table are exactly the property a recursion-friendly system wants.

Scheme	Commit (s)	Open (s)	Verify (ms)	Proof (KB)
Akita (p32 / p64 / p128)	0.185 / 0.126 / 0.346	0.538 / 0.422 / 0.376	12.7 / 10.9 / 10.5	63.1 / 66.7 / 68.3
WHIR (Plonky3)	0.476	6.25	2.1	113.6
Plonky3 STIR	0.674	1.27	3.3	164.9
Binius64 BaseFold	0.215	0.103	0.8	489.2
Flock Ligerito	0.124	0.300	1.1	494.6
WHIR (ProveKit)	1.88	4.11	2.8	563.2
BaseFold (SP1)	2.49	2.30	20.3	933.0

Table 2. Standalone PCS openings at 2^31 bits committed, 8 threads (Akita paper, Table 10)

A crucial caveat accompanies these numbers: Akita's authors argue that with proven soundness - that is, no reliance on conjectured Reed-Solomon proximity gaps - and practical code rates of 1/2 or 1/4, the smallest hash-based proofs (WHIR's) are still "at least 160 KB for non-trivial instances," with asymptotics O(lambda^2 log N + lambda log^2 N / log log N). Magnetar's 598 KiB at 120 proven bits is therefore not an outlier; it sits near the floor of what proven-soundness hash-based proving costs at this scale, and the

Ligerito-in-Flock measurement of 495 KB at 2^31 bits independently corroborates the same regime. This reframes the entire proof-size question: closing on Akita means either attacking the constants of this floor (fewer queries, shorter paths, smaller elements) or shrinking the instance that must pay it - which is precisely what recursion with a low-rate final wrap accomplishes.

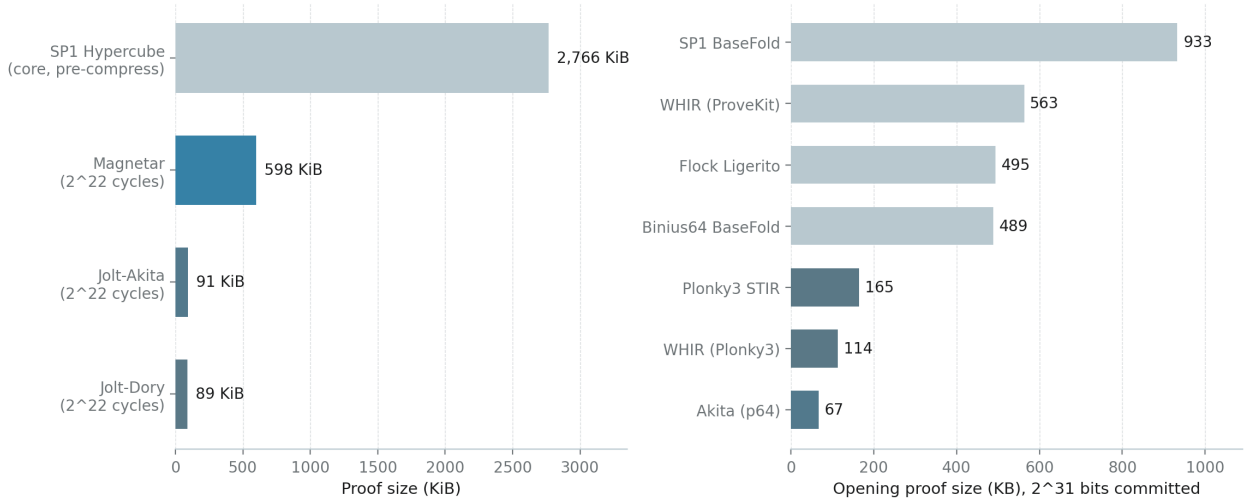


Figure 1. The proof-size landscape. Left: end-to-end zkVM proofs at the 2^22-cycle scale. Right: standalone PCS openings at 2^31 bits committed, from the Akita paper's independent benchmark. Magnetar (blue) inherits the hash-based family's proof-size regime; Akita (steel) defines the lattice frontier.

3. Why Hash-Based Proofs Are Big: A Structural Cost Model

3.1 Anatomy of Magnetar's 598 KiB

Magnetar's protocol is remarkably frugal with everything except Merkle paths. The entire interactive machinery - Virtual Spartan's two outer sum-checks and one inner sum-check, two BinaryTwist instances, BinaryShout, two BinaryRouter instances, MosaicPCS batching and ring switching - communicates only O(log T) field elements per argument. At 2^22 cycles these sum-check transcripts total on the order of 20-40 KiB (tens of rounds times a handful of 16-byte K = F2^128 elements each, across roughly a dozen arguments). The protocol sends exactly one commitment, a single hash root. Everything else - the remaining roughly 550+ KiB - is the single Ligerito evaluation argument that closes the protocol against that commitment. This concentration is good news for an optimizer: one component, one set of parameters, one tradeoff surface.

Ligerito's opening proof has proof size about log(N)^2 / log log(N) up to constants, and its authors measure 255 KiB for a polynomial with 2^24 coefficients over a 32-bit binary field on an M1. In Magnetar's configuration the committed object is the packed witness polynomial over F2^128: every direct witness

segment plus the auxiliary one-hot address components and increments of the memory-checking arguments, all placed on aligned subcubes of one Boolean hypercube by MosaicPCS and packed 128 bits per field element. A useful first-order model for the opening proof is:

Proof ~ Q x log2(codeword) x |digest| + polylog terms

Q = column queries; log2(codeword) = Merkle depth per query; |digest| = hash output size. Only Q, the rate (via the codeword size), and the field/digest sizes are free parameters.

Reading the model against the numbers: a 2^22-cycle execution commits on the order of 2^32-2^33 witness bits (a few hundred committed bits per cycle across the direct segments, the d component polynomials per memory argument, and the increment vectors), which packs to roughly 2^25-2^26 field elements and, at rate 1/2, a 2^26-2^27-entry codeword - Merkle depth 26-27, or about 830-870 bytes per authentication path at 32-byte digests. On the order of six hundred paths then account for the observed ~550 KiB, consistent with Ligerito's measured constant factors. The model exposes the three tunable dials - query count Q, path depth, and element/digest size - and, equally important, their couplings, which Section 5 works through. It also explains the measured growth: from 2^16 to 2^22 cycles, Magnetar's proofs grow by only 219 KiB (377 to 596 KiB) - a nearly flat, logarithmic-ish response that confirms the model and that will repeat at larger traces.

3.2 Why the floor is where it is

The query count Q is set by the soundness budget: to certify that the committed codeword is close to the code (and that the opened evaluations are consistent), random column tests must reject a far codeword except with probability 2^-120. Magnetar insists - uniquely among fast zkVMs - on using only proven Reed-Solomon proximity gaps within the unique-decoding radius, per the Ben-Sasson et al. result it cites. List-decoding regimes (as WHIR and Plonky3-STIR can be configured) shrink proofs substantially, but their analysis rests on conjectured or partially proven proximity gaps that Magnetar's authors explicitly decline to assume. Akita's framing of "at least 160 KB" for proven-soundness hash-based proofs at practical rates is the same observation from the lattice side of the fence. The consequences are stark and worth internalizing before any optimization is priced:

- Rate 1/2 at 120 proven bits is close to the query-efficiency optimum already; lowering the rate buys modest query reduction at the price of a deeper tree and a proportionally more expensive encode and commit phase. The strong rate-versus-size tradeoff belongs to the final wrap, where the instance is small (Section 4), not to the chunk commitment.
- The 128-bit challenge field halves the per-query element count versus smaller fields but doubles element size; 64-bit challenges with more queries can win narrowly on total bytes but lose on verifier hashing - a coupled tradeoff, not a free lunch.

- The single-commitment MosaicPCS design is already the right global move: N separate commitments, each with its own opening argument, would multiply the Merkle-path bill by the number of components. Jagged PCS made the same observation independently, and Magnetar's aligned-subcube layout achieves the batching without Jagged's adapter sum-checks.

One further structural note separates the hash-based and lattice worlds. A lattice commitment is a linear map applied to a small-integer decomposition of the message, so committing to a sparse vector costs only its nonzeros - Akita's pay-per-bit property, worth 32x for bit-vectors versus 32-bit vectors, and tailor-made for Twist and Shout's one-hot addresses. A hash-based commitment must hash the entire encoded codeword, dense or not. Akita's own footnote concedes the partial counterpoint: systematic codes (Brakedown, Bolt) retain the message inside the codeword, so highly repetitive structure - and one-hot rows are maximally repetitive - could in principle be exploited, though no deployed hash-based system does so today. We return to this as a research direction in Section 5.

4. Lever 1: Recursive Composition with Dual-Rate Proving

4.1 The strategy, made precise

Magnetar's paper already sketches its scaling path: prove executions in 2^22-2^24-cycle chunks and recursively compose chunk proofs by proving an implementation of Magnetar's own verifier with BinarySpartan. What the paper leaves as a two-sentence remark deserves to be engineered as the system's central proof-size instrument, because it exploits a structural asymmetry Akita cannot copy cheaply. The verifier of a chunk proof is a small, hash-dominated computation - exactly the workload BinarySpartan is fastest in the world at (BLAKE3 at 872,000 hashes/s, SHA-256 at 401,000 hashes/s including witness generation, on the same class of CPU). Magnetar's authors state the finishing move themselves: "we can produce it at a much lower code rate; a lower rate means fewer queries and hence a smaller proof, and the extra prover work is modest because the computation is small."

Call the two roles the chunk proof and the wrap proof. Chunk proofs are internal artifacts: they never leave the prover's trust domain, so they should be tuned purely for verification speed within the recursion - high code rate (1/2), full 128-bit challenges, whatever makes the guest verifier cheapest per byte checked. The wrap proof is the only artifact the outside world sees: it proves the verifier circuit of the accumulated chunks, a witness vastly smaller than the application witness, and it can afford a low rate (1/8, 1/16, or lower) precisely because its instance is small and its proving happens once. The 598 KiB chunk proof stops being a liability the moment it is only an intermediate; the system's public proof size becomes the wrap size.

4.2 Quantifying the recursion overhead

The per-chunk cost of recursion is the guest-side proving of one verifier invocation. Magnetar's verifier at 2^22 runs 10-12 ms natively, dominated by three workloads: hashing the ~600 KiB transcript (about 9,600 SHA-256 block compressions), checking a few dozen Merkle paths (roughly 60 queries x 27 node hashes), and binary-extension-field arithmetic for the sum-check identities. Guest-side, BinarySpartan proves SHA-256 at 401,000 hashes/s including witness generation, so the transcript hashing costs about 24 ms of proving; adding the field-arithmetic checks and input deserialization, a defensible estimate for one full verifier-guest proving pass is 30-120 ms. Amortized over a 2^22-cycle chunk that costs 612 ms to prove, sequential IVC recursion therefore adds roughly 5-20% prover overhead - and with a binary-tree schedule on 12 cores, near zero wall-clock overhead at the price of about 2x total verifier-proving work.

5-20%

estimated sequential recursion overhead per 2^22 chunk (30-120 ms of verifier-guest proving on a 612 ms chunk); near-zero wall-clock with tree-parallel wrapping

The wrap instance size follows the same arithmetic. A SHA-256 compression circuit carries roughly 25,000-30,000 witness bits, so 9,600 compressions plus path checks plus field identities put the verifier witness at roughly 2^27-2^29 bits - a 16-64x reduction from the chunk's ~2^32-2^33 committed bits. Proving that wrap is sub-second on Magnetar-class hardware. Anchor points for the resulting wrap proof size at low rate, from systems that actually measured small instances: Plonky2 FRI produced 88 KB proofs at 2^27 bits committed; WHIR reports 63 KiB at 100-bit security for degree 2^22 (in its aggressive, not fully proven configuration); and Plonky3 FRI sits at 88-117 KB in the 2^27-2^31 range. A wrap at rate 1/8-1/16 with 120 proven bits plausibly lands at 100-160 KiB. This is the single highest-leverage move available to Magnetar, and it is pure engineering on machinery the paper already specifies.

4.3 Aggregation before the wrap

A refinement multiplies the saving. Rather than wrapping a chain of chunk verifications one at a time, the prover can first batch the k accumulated chunk proofs' final evaluation claims: MosaicPCS batching already reduces many claims on the same committed polynomial to one via a random linear combination, and the same trick applies across proofs - verify a random linear fold of the k proofs' PCS claims in one pass, then wrap once. Because Fiat-Shamir transcripts are sequential data (not sets), this needs no multiset hash - a point of real strategic significance, because ZisK, facing the same distributed-challenge problem, had to adopt LtHash, an SIS-based lattice multiset hash, precisely to parallelize Fiat-Shamir challenge derivation, thereby surrendering the hash-only property Magnetar treasures. Magnetar can keep its purity with a coordinator-star topology: shards compute partial sum-check messages, a single coordinator hashes the round transcript in order, and nothing stronger than SHA-256 is ever invoked.

The competitive reading of this lever: Jolt-Akita is walking the same road in the opposite direction - PR #1911 builds an "Akita field-inline verifier guest" fitting under 2^26 rows, and the Jolt-Akita recursion performance is, per Magnetar's citation of it, "not yet publicly known." A lattice verifier inside a lattice prover must simulate ring arithmetic and norm checks as guest circuits; a hash verifier inside a binary-field prover re-runs SHA-256 in the system that is world champion at proving SHA-256. On recursion workload affinity, the advantage is Magnetar's.

5. Lever 2: PCS-Layer Optimization (Codes, Rates, Fields)

5.1 What Ligerito's flexibility permits

Ligerito is unusually tunable for a deployed PCS: its authors emphasize that any linear code whose generator-matrix rows can be efficiently evaluated drops in - Reed-Solomon, Reed-Muller, and code-switching constructions among them. Two of the three size dials (Section 3) are therefore genuinely open, and a third lever - replacing the code family entirely for speed - is compatible with Magnetar's black-box usage. We price each below; Table 3 summarizes the estimated effects on proof size and proving time at the 2^22-cycle operating point.

Change	Proof-size effect	Prover-time effect	Notes
Rate 1/2 to 1/4	roughly neutral to +10% (deeper paths, slightly fewer queries)	commit phase +60-90% (larger codeword)	soundness slack not convertible to bytes at proven setting
Challenges F2^128 to F2^64 (more queries)	-15 to -25% bytes on elements, +2x verifier hashing	neutral	coupled tradeoff; helps only if verifier hashing is not the bottleneck
Digest 32 B to 16 B (truncated hash)	up to -45% path bytes	neutral	collision budget must be re-derived at 120-bit target
Bolt-style sketched code for encoding	slightly larger (+10-20%)	commit 3-4x faster	geared to speed; Bolt reports 3-4x faster commitment than RS at N = 2^30 over GF(2^32)
Lower rate only on final wrap	wrap-only: fewer queries on small instance	wrap-only, modest	the dual-rate strategy of Section 4; does not touch chunk proofs

Table 3. Estimated effects of PCS-layer parameter changes at 2^22 cycles (model-based estimates)

5.2 Code switching for commitment speed

The Bolt paper (2026) introduces sketched-code commitments whose encoding costs about (3+epsilon)N field additions plus a Merkle hash of (1+epsilon)N elements - versus 8N additions and a 4N Merkle hash for Blaze and substantially more work for Reed-Solomon encoding - and reports 3-4x faster commitment than RS-based schemes for N = 2^30 over GF(2^32), with proofs 5-15x smaller than Brakedown's in its fast variant (about 2 MB in the slower, size-optimized variant). For Magnetar, whose commit phase (encode + Merkle hashing of the packed witness) is a major share of its 612 ms chunk proving time, adopting a Bolt-class code inside Ligerito is the single largest credible proving-speed lever at constant security: the interface is identical, the witness is bit-packed exactly as today, and the cost lands where BinarySpartan is already fastest. The honest price: slightly larger opening proofs, which the dual-rate strategy (Section 4) renders irrelevant for the public-facing artifact.

5.3 Sparsity-aware commitment of one-hot data

The most speculative but intellectually interesting PCS direction exploits structure Magnetar already has. The one-hot address components that BinaryTwist and BinaryShout commit are concatenations of standard basis vectors - each row has exactly one set bit - and the increment vectors are similarly sparse or low-entropy. A systematic code retains the message block inside the codeword, so the committed codeword is mostly zeros or repeated patterns; a Merkle tree over such a codeword contains massive repeated subtrees, and a commitment procedure that caches distinct subtrees could hash far fewer nodes than the codeword length. This is the hash-based shadow of Akita's pay-per-bit argument, acknowledged as unexplored even by the lattice camp (Akita's own footnote on systematic codes). We flag it as a research item rather than a roadmap line: it would need a soundness proof that repeated-subtree skipping does not weaken binding (the salted-leaf structure must be preserved), and its payoff scales with the one-hot share of the witness, plausibly 20-40% of committed bits in memory-heavy workloads.

5.4 Alternative backends and their caveats

Two alternative hash-based backends deserve mention for completeness. FRI-Binius (Diamond-Posen, EUROCRYPT 2025) contributes the ring-switching technique Magnetar already uses and, on a characteristic-2 adaptation of BaseFold, an extremely fast tiny-field multilinear PCS - but its proofs at scale sit in the same hundreds-of-KB regime (Binius64 BaseFold measures 489-690 KB in Akita's benchmark), so switching would trade like for like. WHIR-class Reed-Solomon testing achieves the smallest hash-based proofs known (63-114 KB) but only in list-decoding configurations whose proximity analysis Magnetar's proven-soundness stance rules out; its unique-decoding configuration measured 691 KB at 2^35 bits. The consistent conclusion: within the proven-soundness, practical-rate regime, no hash-based backend materially beats Ligerito on size. The winning moves are the rate schedule (cheap, wrap-only) and shrinking the instance that pays the floor - that is, recursion.

6. Lever 3: Protocol-Layer Size Reduction

6.1 Commit fewer bits

Every bit Magnetar commits lands in the codeword, deepens the tree, and broadens the query bill - so the cheapest bit is the one never committed. Virtual Spartan is already the state of the art on this axis: register values, RAM words, decoded instruction fields, and shift outputs are read by the R1CS but never committed, their evaluation claims discharged instead by BinaryTwist, BinaryShout, and BinaryRouter. The remaining committed inventory at 2^22 is dominated by four families: the direct datapath segments, the d one-hot component polynomials per memory argument (two reads plus one write address set), the write-increment vectors, and the small direct copies that BinaryRouter consumes. Each family has known, quantifiable slack.

- Address-component width d: BinaryTwist commits d components of width K^(1/d) per address. For the register file (K = 2^5 registers) d = 1 already minimizes bits; for RAM (K up to 2^32 or more) larger d shrinks components superlinearly but multiplies their count and adds address-virtualization sum-check rounds - the paper leaves the operating point to engineering, and the committed-bit budget is the right objective function for choosing it per workload.
- Jagged PCS comparison: Jagged's independently developed design commits the entire trace as one polynomial with no adapter sum-checks and verifier cost of 5 field multiplications per trace element. MosaicPCS matches the single-commitment property with an even cheaper claim-lift (free suffix append). The Jagged paper's most recursion-relevant idea - implementing the verifier as a circuit whose size depends only on total trace area, not per-table structure - is worth adopting verbatim for the wrap circuit, since it is what keeps the wrap witness near 2^27-2^29 bits.
- Sum-check message hygiene: the outer sum-checks bind instance variables (log q rounds) and row variables (log m rounds) with degree-2/3 messages over F2^128. The 2024-2025 optimization literature (univariate skip, move-to-front, and the eq-optimization) already compresses these transcripts substantially inside BinarySpartan; at 20-40 KiB total they are a secondary target, worth perhaps 10-20 KiB - real but not where the battle is.

A different, underexplored protocol lever is deferring commitment of the one-hot components entirely. Because BinaryTwist's one-hotness check is itself a sum-check over committed component polynomials, a design that replaces committed components with virtual ones - discharged through routings or through the bytecode argument - would eliminate the single largest auxiliary commitment family. We estimate the ceiling here at 15-30% of committed bits, but the protocol redesign cost is high and the soundness story (one-hotness must be checkable somewhere) is the binding constraint. It is a paper-worthy extension, not an engineering chore.

6.2 Zero-knowledge as a cheap add-on

Both systems currently lack ZK, and Magnetar's path to it is routine for hash-based commitments: random masking rows in the codeword and blinding the sum-check openings, standard since Ligero and Brakedown. The cost is a small constant factor in proof size and negligible prover time - worth doing early, since it is cheap insurance for the private-input workloads (proof-of-possession, private smart contracts) that Akita also lists as future work, and since retrofitting ZK after the recursion architecture is set is strictly harder.

7. Lever 4: Proving Throughput (Cycles per Second)

Magnetar's proving stack is BinarySpartan, and its measured envelope - 6.5-7.7M cycles/s at 2^22 on 12 CPU cores - already clears the field. The BinarySpartan paper's own numbers define the headroom: BLAKE3 at 872,000 hashes/s, SHA-256 at 401,000 hashes/s, and Keccak-f at 287,000 permutations/s including witness generation, all CPU-only, with the authors explicitly noting the absence of GPU/Metal acceleration. Four levers, in descending order of confidence, project the next factor.

Lever	Mechanism	Estimated effect	Confidence
Small-domain sum-check arithmetic (2025/1117)	replace large-field multiplications with small-domain ops where operands are bits or subfield elements; plus eq()-factor optimization	1.3-2x on sum-check-dominated phases (Jolt measured 2-3x; BinarySpartan already integrates part)	High - published, implemented in Jolt
Bolt-class sketched codes in Ligerito	encoding cost (3+e)N additions + (1+e)N Merkle hashes vs RS encoding	commit phase 3-4x faster; end-to-end 1.5-2.5x if commit is 50-70% of chunk time	Medium-high - Bolt benchmarks are measured; integration is engineering
Streaming small-space prover (2025/611-style)	stream trace contributions through sum-check rounds; no recursion needed for memory	memory 1.5 KiB/cycle to O(100) B/cycle; runtime overhead below 2x; unlocks 2^26+ monolithic or cheaper chunking	Medium - proven for Jolt's architecture; adaptation required
GPU/Metal offload	hashing, encoding, and Merkle tree building are throughput-shaped workloads	3-10x on commit-dominated phases (industry-typical for SHA/Keccak pipelines)	Speculative - unmeasured for this stack

Table 4. Proving-throughput levers with confidence and estimated effect

The small-domain optimization deserves a precise note because Magnetar is unusually well positioned for it: its witnesses are bits, and the 2025 sum-check paper's core trick - recognizing when products of multilinear terms live in a small subfield and computing them with small-field arithmetic - applies directly to the F2-coefficient relation's sum-checks and to eq(r,x) factors with Boolean x. The same paper's eq-optimization

cut memory pressure up to 20x when baselines approach their limits, which matters at 2^24-cycle chunks where Magnetar's 25 GiB footprint is the binding constraint. The packed-sumcheck line (Wei et al., cited in the Akita paper) extends the same idea to small characteristic, i.e., binary fields natively.

Streaming completes the picture. Nair, Thaler, and Zhu's small-space Jolt prover demonstrates that a sum-check zkVM can stream trace contributions across rounds with overhead "potentially well below a factor of two" and no recursion at all; Magnetar's single-shared-witness architecture is arguably even better suited to it, since its two SIMD relations already read one witness matrix row-block at a time. The strategic value is not speed but reach: today Magnetar stops at 2^24 (25 GiB), while Jolt-Akita runs 2^27 within 35 GiB; streaming erases that asymmetry without waiting for the recursion milestone, and the two techniques compose - stream within chunks, recurse across them.

10-20M cycles/s

plausible Horizon-1 combined throughput on the same 12 CPU cores (small-domain arithmetic + Bolt-class codes), with GPU headroom beyond

8. The Folding, IVC, and PCD Landscape for a Hash-Only zkVM

8.1 Why classical folding does not apply - and what does

Folding schemes are the dominant paradigm for recursive proving in the curve world, and it is tempting to ask why Magnetar does not simply fold. The structural answer: folding reduces two instances of a relation to one by taking random linear combinations of their witnesses and commitments, which requires the commitment scheme to be linearly homomorphic. Pedersen-style commitments (Nova, SuperNova, HyperNova, ProtoStar, ProtoGalaxy, CycleFold) and Ajtai commitments (LatticeFold, Neo, SuperNeo, ProtogaLattice, PikkuFold) are; Merkle-tree commitments are not - there is no meaningful linear combination of two hash roots. This is why every hash-based system in production (RISC Zero, SP1, ZisK, and Magnetar's plan) recurses by proving the verifier circuit instead. The one counterexample in the literature, Arc (hash-based folding), pays for it with large verifier circuits, per the SuperNeo paper's comparison - the opposite of what a recursion-friendly system wants.

System	Year / venue	Security base	Recursion overhead	Proof size
Nova	CRYPTO 2022 (2021/370)	discrete log	2 group scalar mults (~10K gates)	O(|F|) group elts; O(log|F|) compressed
SuperNova	2022/1758	discrete log	non-uniform IVC: per-instruction circuits	as Nova

System	Year / venue	Security base	Recursion overhead	Proof size
HyperNova	CRYPTO 2024	discrete log	one sum-check per step	as Nova-class
ProtoStar	2023/620	discrete log	k+2 EC mults, no FFT/setup	O(d*) field elts
ProtoGalaxy	2023/1106	discrete log	log field ops + O(1) hashes marginal	amortized constant per folded instance
CycleFold	2023/1192	discrete log	1 scalar mult on 2nd curve (~1-1.5K gates)	enables binary-tree IVC
LatticeFold	2024/257	Module-SIS	sum-check-based norm control	~HyperNova performance claimed
Neo / SuperNeo	2025/294, 2026/242	Module-SIS	pay-per-bit; small fields; non-SIMD (SuperNeo)	six-property frontier
ProtogaLattice	2026/1317	lattice	sum-check-free verifier circuits	8.13 KB per fold step at 2^30
PikkuFold	2026/1809	lattice	layered random projections	5.7 KB/step (vs Cyclo 30 KB, SALSAA 60 KB)
Magnetar (plan)	2026-2423	hash-only	verifier-as-guest: 30-120 ms proving per 2^22 chunk	598 KiB today; wrap 100-160 KiB projected

Table 5. The folding and recursive-composition landscape, 2021-2026

Two rows of Table 5 deserve emphasis because they define the frontier Magnetar's plan competes with. ProtogaLattice and PikkuFold have collapsed the per-step communication of lattice folding to 5.7-8.13 KB - a 4-10x improvement over the 2024-2025 lattice-folding state of the art - by attacking exactly the cost centers the earlier schemes tolerated: ProtogaLattice eliminates sum-check verifier circuits (whose random-oracle invocations dominate recursive overhead), and PikkuFold removes in-protocol commitments entirely via certified Johnson-Lindenstrauss projections. If lattice IVC systems settle at single-digit-KB per step, an IVC chain of 2^20 steps still carries 5-8 GB of accumulated structure before compression - so the end-to-end race remains between "fold cheaply, compress expensively" and "verify cheaply, wrap once." Magnetar's verifier-as-guest sits in the second family, and its 10-12 ms, hash-dominated verifier is the cheapest guest in the field.

8.2 IVC architecture decision for Magnetar

For a hash-only system the design space collapses to three workable schedules. Sequential IVC: chunk i's proof absorbs the accumulator ("all prior chunks verified") into chunk i+1's public input; overhead 5-20% per chunk (Section 4), single-threaded, minimal total work. Binary-tree IVC (the schedule CycleFold popularized in the curve world): pairs of proofs combine in a tree of depth log k; wall-clock parallelism at about 2x total verifier work; final proof independent of k. Aggregation-then-wrap: batch the k chunk proofs' evaluation claims by random linear combination into one claim, prove the batch-verification circuit once. The correct default for Magnetar is tree-IVC with claim-batching at each internal node, because both mechanisms reuse MosaicPCS machinery it already has, and because tree schedules degrade gracefully when

chunks arrive out of order - the property distributed proving actually needs.

8.3 PCD and the distributed Fiat-Shamir problem

Proof-carrying data - the Valiant/Bitansky lineage Magnetar cites for succinct blockchains - falls out of tree recursion for free: each node of a computation DAG carries a proof that its inputs' proofs verified. The unsolved engineering question is distributed Fiat-Shamir across shards: challenges depend on all provers' partial messages, so either a coordinator serializes transcript hashing (star topology; latency-bound but hash-only and sound), or shards derive challenges from an aggregate of partial commitments - which is exactly where ZisK adopted LtHash, an SIS-based multiset hash, and thereby left the hash-only regime. The hash-only alternatives are a Merkle-tree transcript (still sequential hashing, but parallelizable path verification) or a two-round interaction. None is free; all are sound; the report's recommendation is the coordinator-star for v1 with the Merkle-transcript as the scale-up path, and explicit framing that this is a throughput-latency tradeoff, not a security one.

9. The Hybrid Option: Adding a Lattice Layer

The most direct route to Akita-class proof sizes is, of course, Akita itself: a hybrid Magnetar that keeps its binary-field arithmetization and sum-check machinery but swaps the commitment layer - or only the final wrap - to a lattice PCS. The gains are real and quantified: 61-70 KB standalone openings at 2^30-sized polynomials, sub-20 ms verification, pay-per-bit commitment that would slash the cost of the one-hot address components (the sparsest, most-committed data Magnetar carries), and 128-byte commitments. Jolt's integration data shows the swap is system-level feasible: Jolt-with-Akita proved 1.3-2.2x faster and verified 2.2-7.4x faster than Jolt-with-Dory with proof sizes within 9% - evidence that a lattice PCS slots under a sum-check zkVM without breaking it.

The costs are equally concrete, and they are philosophical before they are technical. Magnetar's identity is the three-property trifecta - general-purpose, hash-only, recursion-friendly - and a lattice layer forfeits the middle property, reclassifying the system alongside SP1 Hypercube and ZisK as hash-and-more. Akita's open items transfer with the swap: QROM analysis of Fiat-Shamir is future work, ZK is future work, and the Module-SIS parameter regime (like all lattice cryptography) tracks an evolving cryptanalysis literature in a way SHA-256 does not. There is also an engineering asymmetry: Akita's prover is its own bottleneck (its paper concedes proving time lags Greyhound and RoKoKo; opening at 2^35 bits takes 16-19 s), so a hybrid inherits the commitment-layer cost profile that Magnetar's speed advantage currently exists to avoid.

Architecture	Final proof size	Security story	Prover speed	Cost to reach
Pure Magnetar + dual-rate wrap (Horizon 2)	100-160 KiB (est.)	hash-only, 120-bit proven, unique-decoding only	unchanged + 5-20% recursion overhead	engineering on specified machinery
Hybrid: lattice final wrap only	65-90 KiB (est.)	hash chunks, lattice wrap: mixed assumptions	wrap gains Akita opening cost (sub-second)	moderate: one PCS integration + its QROM/ZK caveats
Full Akita backend swap	61-98 KB (measured class)	Module-SIS throughout; ROM-FS, QROM open	unknown for Magnetar's arithmetization; Akita prover is the slow layer	large: re-derive the whole commitment stack

Table 6. The hybrid decision matrix

Our assessment: the hybrid wrap is the rational fallback if Horizon 2 misses its 100-160 KiB projection - it concedes the purity principle only for the smallest artifact in the system while keeping the proving engine hash-based and fast. The full backend swap is strategically unnecessary: it answers a size race Magnetar can otherwise tie, at the cost of the differentiated security property that justifies Magnetar's existence. The honest framing is that this is a philosophy-versus-pragmatism decision that belongs to Magnetar's maintainers and their users - Ethereum's post-quantum roadmap, which BinarySpartan explicitly targets, currently pays for provable-soundness conservatism, and that constituency is exactly Magnetar's natural market.

10. Quantified Roadmap and Competitive Outlook

Figure 2 assembles the three horizons against Akita's reference lines. The projections use the cost model of Section 3, the recursion arithmetic of Section 4, and the measured anchor points catalogued throughout; ranges reflect parameter uncertainty, not optimism. Horizons are cumulative and largely independent, so partial delivery still moves the needle.

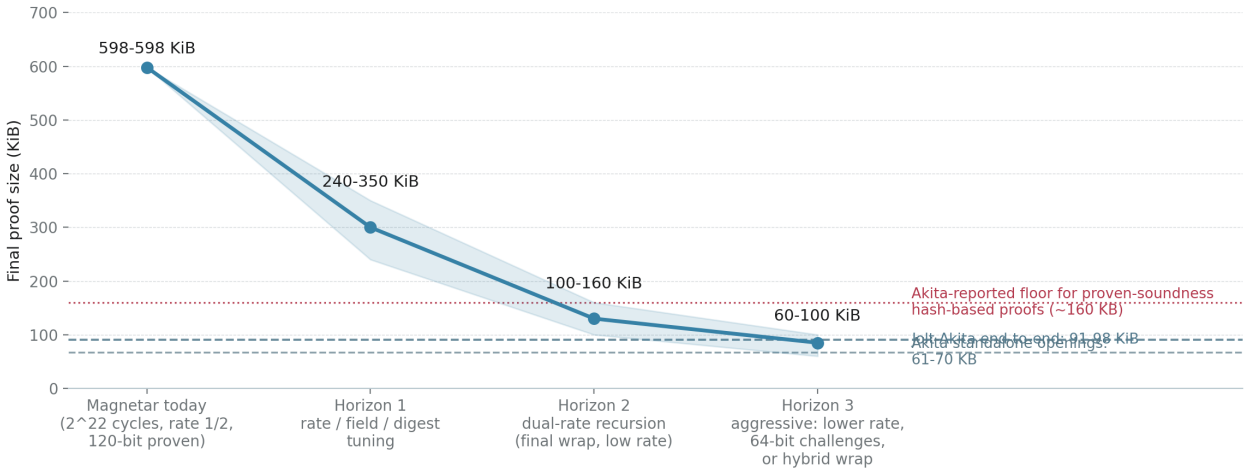


Figure 2. Magnetar proof-size roadmap (blue, with estimate ranges) against Akita reference lines: Jolt-Akita end-to-end proofs (91-98 KiB), Akita standalone openings (61-70 KB), and the Akita-reported floor for proven-soundness hash-based proofs near 160 KB. Horizon bars are model-based estimates, not measurements.

Horizon	Actions	Final proof	Throughput	Traces supported
H1: parameterize (months)	rate/digest/field tuning; small-domain sum-check; Bolt-class codes; ZK masking	240-350 KiB	10-20M cycles/s	2^22-2^24 chunks
H2: recurse (1-2 quarters)	verifier-guest + tree IVC + claim batching; low-rate wrap; streaming within chunks	100-160 KiB wrap	6.9M+ x (1 - 0.05..0.20) amortized	unbounded via chunks
H3: push (research)	64-bit challenge fields; systematic-code sparsity; hybrid lattice wrap if needed	60-100 KiB	GPU-dependent	unbounded

Table 7. Three-horizon roadmap (all size/throughput figures beyond today are estimates)

10.1 Risk register

- Proven-soundness floor risk: if fewer-query regimes genuinely require list-decoding conjectures, H2's 100-160 KiB is the honest limit of pure construction, and sub-100 KiB requires H3's 64-bit challenges (a coupled verifier cost) or the hybrid wrap. Mitigation: the floor applies to the wrap instance only; its small size is the hedge.
- Wrap-witness growth risk: the verifier transcript grows with chunk proof size, so a lazy H1 (bigger chunk proofs) inflates the wrap witness and can offset H1 gains in the final artifact. Mitigation: batch claims before wrapping (Section 4) and keep chunk rate high.
- QROM exposure: like every 2026 system, Fiat-Shamir is argued in the classical ROM; a quantum adversary argument for the full composed protocol is open for both Magnetar and Akita. Not a differentiator, but a shared debt worth paying early on the recursion milestone, since the wrap is where the composition argument concentrates.

- Engineering risk: streaming and tree-IVC touch the prover's core loops; the small-space Jolt experience (sub-2x overhead, no recursion) suggests feasibility but the estimate carries integration risk typical of second-order refactors.
- Competitive motion: Akita's stated roadmap targets smaller proofs and faster proving; Jolt's recursion PR is already merged-in-progress. The 5-18x proving lead has room to erode; the hash-only security property cannot be replicated by Akita at any speed.

10.2 The verdict

On today's evidence, Magnetar is already the stronger system on three of five axes - proving throughput by 5-18x, verification at operating point, and assumption quality for the conservative buyer - while losing proof size by 6.6x and tying on memory-with-chunking. Horizon 2, built entirely from machinery the Magnetar paper already specifies, projects final proofs of 100-160 KiB: parity in bytes-per-proof with Jolt-Akita's under-100 KB within a factor of 1.2-1.7, at roughly 5-9x the proving speed and with the strictly stronger (proven, hash-only) security story. That configuration - not proof-size victory, which likely requires the hybrid - is the realistic winning position: fastest prover, comparable proof, cleanest assumptions. Akita's irreducible counter-advantage is pay-per-bit commitment for sparse polynomials, which no amount of Merkle-path engineering fully matches; it keeps Akita the right choice where commitment sparsity or sub-70 KB proofs dominate the requirements.

11. Conclusions and Recommendations

The deep research supports a clear build order. First, instrument the cost model: measure the committed-bit inventory and the query/path/digest split of the 598 KiB directly, so every subsequent lever lands on measured constants rather than estimates - the model in Section 3 makes this a one-week measurement task with the existing Rust implementation. Second, ship the sum-check and code-layer speed work (small-domain arithmetic, Bolt-class codes): it is published, mechanical, and compounds with everything else. Third, build the verifier-guest and the low-rate wrap as the flagship milestone, with claim-batching and tree-IVC as the scaling story; this is where the proof-size question is actually decided, and where Magnetar's hash-verifier affinity is a structural advantage over a lattice verifier inside a lattice prover. Fourth, adopt streaming inside chunks to erase the 2^24 memory ceiling independently of the recursion milestone. Fifth, add ZK masking early and cheaply. The hybrid lattice wrap remains the documented fallback - a decision to be taken only if the measured wrap misses the 160 KiB line that the proven-soundness literature implies.

The broader lesson of the 2024-2026 literature is that the folding revolution, however luminous, is a homomorphic-commitment revolution: its techniques transfer to hash-based systems only as inspiration (tree schedules, non-uniform proving, batch-then-compress), never as drop-ins. Magnetar's correct strategy is therefore the one its paper already gestures at - recursion native to its own verifier's strengths - executed with the same engineering discipline that took BinarySpartan to the top of the hash-proving benchmarks. The

competitive question versus Akita then resolves not into who wins a single metric, but into which bundle of properties the market values: Magnetar projects to hold the bundle {fastest prover, comparable proof size after recursion, proven hash-only security}, and Akita holds {smallest proofs, pay-per-bit commitments, lattice-assumption agility}. Both are defensible positions; only one of them is Magnetar's to lose.

Appendix: Sources and Attribution

Primary sources, with IACR ePrint numbers where available. Measured figures in this report are attributed to these papers; projections and estimates are the report's own, with assumptions stated inline.

- Setty, S.: Magnetar: High-speed post-quantum RV64 zkVM from standard hashes. Manuscript 2026-2423 (October 2026). All Magnetar measurements: Tables 2-7 of that paper.
- Dao, Q., Bodaghi, O., Khajehpour, A., Vitto, G., Badakhshan, M., Georghiades, M., Liu, F., Zhang, J., Thaler, J.: Akita: A high-performance lattice-based polynomial commitment scheme. LayerZero Labs / a16z crypto research (2026). All Akita and Jolt-Akita measurements, the hash-PCS benchmark of Table 2, and the 160 KB proven-soundness floor claim.
- Setty, S.: BinarySpartan: Spartan over binary fields. ePrint 2026/1656. Hash-proving rates (BLAKE3 872K/s, SHA-256 401K/s, Keccak-f 287K/s) and Plonky3/Binius64/Hashcaster comparisons.
- Novakovic, A., Angeris, G.: Ligerito: A small and concretely fast polynomial commitment scheme. ePrint 2025/1187. Proof-size asymptotics and the 255 KiB / 2^24-coefficient measurement.
- Setty, S., Thaler, J.: Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. ePrint 2025/105, CRYPTO 2026.
- Diamond, B.E., Posen, J.: Succinct arguments over towers of binary fields (Binius). ePrint 2023/1784; Polylogarithmic proofs for multilinears over binary towers (FRI-Binius / ring switching). ePrint 2024/504, EUROCRYPT 2025.
- Hemo, T., Jue, K., Rabinovich, E., Roh, G., Rothblum, R.D.: Jagged polynomial commitments (or: how to stack multilinears). ePrint 2025/917.
- Gurkan, K., Novakovic, A., Rothblum, R.D.: Bolt: Faster SNARKs from sketched codes. ePrint 2026/310.
- Kallesoe, A., Khoshakhlagh, H.: Grand Danois: Succinct multilinear polynomial commitments over lattices. ePrint 2026/1196. Nguyen, N.K., O'Rourke, G., Zhang, J.: Hachi. ePrint 2026/156. Klooß, M., Lai, R.W.F., Nguyen, N.K., Osadnik, M., Tucci, L.: RoKoKo. ePrint 2026/575.
- Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive zero-knowledge arguments from folding schemes. ePrint 2021/370, CRYPTO 2022. Kothapalli, A., Setty, S.: SuperNova. ePrint 2022/1758. HyperNova: CRYPTO 2024.

- Bünz, B., Chen, B.: ProtoStar. ePrint 2023/620, ASIACRYPT 2023. Eagen, L., Gabizon, A.: ProtoGalaxy. ePrint 2023/1106. Kothapalli, A., Setty, S.: CycleFold. ePrint 2023/1192.
- Boneh, D., Chen, B.: LatticeFold. ePrint 2024/257. Nguyen, W., Setty, S.: Neo. ePrint 2025/294; Neo and SuperNeo. ePrint 2026/242. Balbás, D., Nitulescu, A., Plançon, M.: ProtogaLattice. ePrint 2026/1317. Osadnik, M.: PikkuFold. ePrint 2026/1809.
- Bagad, S., Dao, Q., Domb, Y., Thaler, J.: Speeding up sum-check proving. ePrint 2025/1117. Dao, Q., Thaler, J.: More optimizations to sum-check proving. ePrint 2024/1210.
- Nair, V., Thaler, J., Zhu, M.: Proving CPU executions in small space. ePrint 2025/611. Arnon, G., Chiesa, A., Fenzi, G., Yogev, E.: WHIR. ePrint 2024/1586.
- Yang, Y., Cheng, Y., Tang, H., Yang, G., Zhang, B., Ren, K.: SoK: Understanding zkVM: from research to practice. ePrint 2026/525, AsiaCCS 2026. Jolt repository, PR #1911 (Akita field-inline verifier guest). ZisK: Secure challenge derivation in ZisK (LtHash), zisk.technology (2025).

Conventions: KiB denotes 1024-byte units as in the Magnetar paper; KB denotes decimal kilobytes as in the Akita paper; the roughly 2.4% difference is immaterial to every comparison drawn. Superscript notation 2^22 denotes two to the 22nd power throughout.